TP MongoDB

Construire et interroger la base d'une mini-boutique en ligne « shop_tp »

1. Objectifs du TP

- Créer une base MongoDB, ses collections et un jeu de données réaliste.
- Interroger les données : filtres, projections, tri, opérateurs logiques, tableaux et expressions régulières.
- Modifier les documents : `updateOne`, `updateMany`, `replaceOne`, opérateurs sur les tableaux.
- Optimiser avec les index et analyser les requêtes avec `explain()`.
- Produire des indicateurs avec le pipeline d'agrégation, puis sécuriser la structure avec une validation JSON Schema.

2. Prérequis (au choix)

- **mongosh** installé localement (avec MongoDB ou en shell autonome) ;
- ou **MongoDB Compass** (interface graphique) ;
- ou un cluster gratuit **MongoDB Atlas** (M0).

Si vous utilisez Atlas, créez un cluster gratuit puis un utilisateur de base de données.

3. Contexte et jeu de données

Vous allez construire la base d'une mini-boutique en ligne composée de trois collections, décrites ci-dessous.

3.1. Collection customers (10 documents)

Les personnes qui achètent dans la boutique : identité, contact, plusieurs adresses, niveau et points de fidélité, date de création.

Champ	Type	Description / contrainte
_id	int	Clé primaire
name	string	Nom complet
email	string	Unique
age	int	≥ 0
addresses	tableau d'objets	Chaque adresse contient `type` (« domicile » ou « livraison »), `city` (string) et `country` (string, ex. « MA »)
loyalty	objet	`tier` ∈ {Bronze, Silver, Gold}, `points` (int ≥ 0)
createdAt	date	Date de création du compte

3.2. Collection products (10 documents)

Le catalogue des articles pouvant être commandés : catégorie, prix, stock, mots-clés et note.

Champ	Type	Description / contrainte
_id	int	Code produit
name	string	Nom du produit
category	string	Ex. Électronique, Mobilier, Accessoires, Papeterie
price	int / double	≥ 0 (en MAD)
stock	int	≥ 0 (quantité disponible)
stockMin	int	≥ 0 (seuil en dessous duquel il faut réapprovisionner)
tags	tableau de string	Mots-clés
rating	double	Note de 0 à 5

3.3. Collection orders (12 documents)

Les achats des clients. Chaque commande référence un client, contient des lignes d'articles (produit, quantité, prix unitaire), un statut, une ville de livraison, une date de création et un total enregistré.

Champ	Type	Description / contrainte
_id	int	Numéro de commande
customerId	int	Référence vers `customers._id` (non imposée par MongoDB : c'est à vous et à vos pipelines de la garantir)
status	string	Une valeur parmi : PENDING, PAID, CANCELLED, REFUNDED
items	tableau d'objets	`productId` (int, référence vers `products._id`), `qty` (int ≥ 1), `price` (prix unitaire ≥ 0 au moment de la commande, pour garder l'historique)
createdAt	date	Date et heure de la commande
shippingCity	string	Ville de livraison
total	int / double	≥ 0, doit être égal à Σ qty × price

4. Mise en place

4.1. Créer la base « shop_tp »

Dans mongosh, placez-vous sur la base `shop_tp` (elle sera réellement créée à la première insertion).

4.2. Créer les collections

Première étape, avant toute insertion. Les contraintes à respecter sont celles des tableaux de la partie 3.

4.2.1. Collection customers

Vous devez créer la collection **d'abord sans contrainte.** Créez-la vide, sans validation :

```
db.createCollection("customers")
```

Ajouter ensuite les contraintes à customers. Avec `collMod`, ajoutez un validateur `$jsonSchema` qui respecte le tableau de la collection :

```
db.runCommand({ collMod: "customers",
  validator: { $jsonSchema: {
    bsonType: "object",
    required: [ ... ],
    properties: {
      age: { bsonType: "int", minimum: 0 },
      ...
    }
  } }
})
```

4.2.2. Collections products et orders:

Pour les deux collections products et ordes il faut les créer avec leurs contraintes dès la création. Créez-les directement avec leur validateur :

```
db.createCollection("products", {
  validator: { $jsonSchema: { ... } }
})
```

- **products** : `price`, `stock` et `stockMin` ≥ 0 ; `tags` est un tableau de chaînes ; `rating` entre 0 et 5 ;
- **orders** : `status` ∈ {PENDING, PAID, CANCELLED, REFUNDED} ; `items` est un tableau **non vide** dont chaque élément a `productId`, `qty` ≥ 1 et `price` ≥ 0 ; `total` ≥ 0.

4.3. Insérer les documents

Insérez 10 clients, 10 produits et 12 commandes respectant la structure décrite ci-dessus.

Lors des insertions il faut respecter les éléments suivants :

- au moins un client ayant une adresse de **livraison à Rabat**, et un autre ayant une adresse **à Rabat (domicile)** et une adresse de **livraison dans une autre ville** ;
- quelques produits dont le `stock` est **inférieur** à leur `stockMin` ;
- une commande dont le `total` est volontairement **erroné** (pour la dernière partie de la question 12)
- une commande **5011** au statut PENDING, un produit **104** sans le mot-clé « ergonomique » et un produit **109** avec le mot-clé « papier ».

5. Questions

1. **Créer et lister** les bases et les collections (`show dbs`, `use shop_tp`, `show collections`). Vérifiez que `customers`, `products` et `orders` contiennent 10, 10 et 12 documents.
2. **find() :** affichez tous les clients de niveau Gold.
3. **Projection :** affichez uniquement `name`, `email` et `loyalty.points` des clients Silver, sans `_id`.

4. **sort(), limit(), skip() :** listez les produits par prix décroissant et affichez les 3 premiers. Ensuite, ignorez le premier et récupérez les 2 suivants.
5. **Comparaison :** trouvez les clients de 30 ans ou plus ayant plus de 1000 points.
6. **Opérateurs logiques :** trouvez les commandes au statut PAID ou REFUNDED.
7. **$elemMatch :**
   - trouvez les clients qui ont une adresse à Rabat, avec la notation pointée ;
   - trouvez ensuite ceux qui ont une adresse **de livraison** à Rabat, avec la notation pointée puis avec `$elemMatch` ;
   - quel client apparaît à tort avec la notation pointée, et pourquoi ?
8. **Expression régulière :** trouvez les produits dont le nom contient « bureau », sans tenir compte de la casse.
9. **Mises à jour :** augmentez de 50 les points des clients Silver, puis passez la commande 5011 de PENDING à PAID.
10. **Remplacement ou $set ?** Remplacez le client `_id: 6` par un nouveau document, puis comparez avec un `$set` qui ne modifie que les points. Que constatez-vous ?
11. **Tableaux :** ajoutez le mot-clé « ergonomique » au produit 104 sans doublon, puis retirez « papier » du produit 109.
12. **$expr :**
    - Trouvez les produits dont le stock est inférieur à leur `stockMin` ;
    - essayez d'abord `{ stock: { $lt: "$stockMin" } }` : que renvoie cette requête, et pourquoi ?
    - Trouvez les commandes dont le total enregistré ne correspond pas à la somme des qty × price.
13. **Index simple :** créez un index unique sur `customers.email`.
14. **Index composé et explain() :** créez `{ status: 1, createdAt: -1 }` sur `orders`, puis vérifiez avec `explain()` qu'il est utilisé par une requête qui filtre par statut et trie par date.
15. **Agrégation :** pour chaque catégorie de produits, calculez le nombre d'unités vendues et le chiffre d'affaires.
16. **Agrégation :** trouvez les 3 meilleurs clients selon le chiffre d'affaires des commandes PAID.
17. **Recherche textuelle :** ajoutez un index texte sur `products.name` et `products.tags`, puis cherchez les produits liés au mot « bureau ».
18. **Validation :** sur `orders`, garantissez que `total` est un nombre et que `items` est un tableau non vide. Testez avec une commande invalide.
19. **$set, $unset, $rename :** ajoutez `vip: true` aux clients Gold, supprimez ensuite ce champ, puis renommez `rating` en `stars` dans `products`.